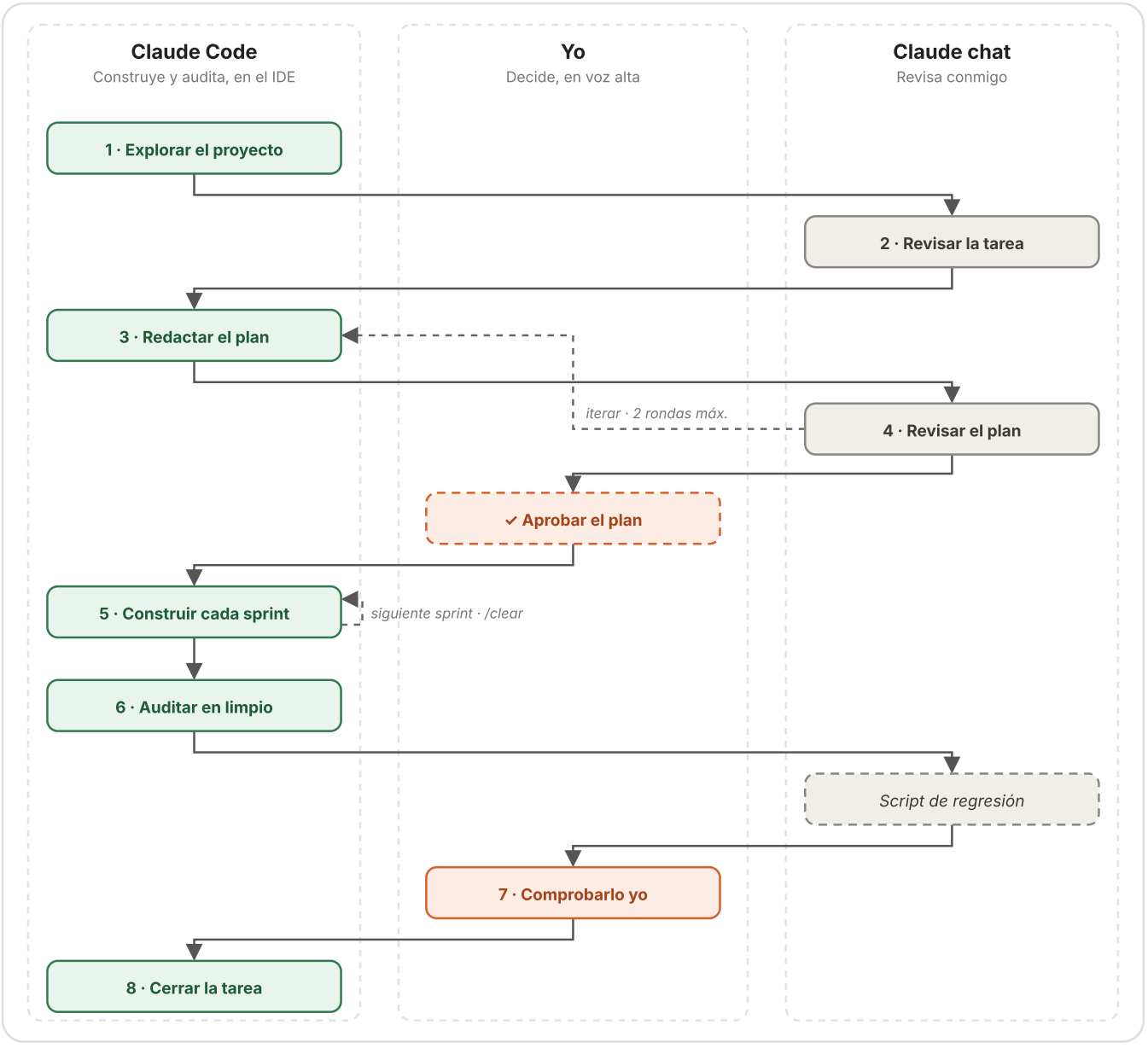
Especificar, construir, verificar

Claude escribe el código. Yo soy el dueño de la especificación, las decisiones y la evidencia.

Hay dos puntos de control humanos que nunca me salto: aprobar el plan antes de construir y aprobar el resultado antes de fusionar. Quien construye nunca se evalúa a sí mismo: Claude Code construye, una sesión limpia de Claude Code audita, un chat de Claude revisa conmigo y la comprobación final la hago yo. Un harness de hooks y skills convierte el método en reglas que el agente no puede saltarse.



Paso a paso

1 Explorar el proyecto `Claude Code · modo plan · /explore`

- Si no hay Git, crea un repositorio local con un commit `baseline` y pregunta si crear un repositorio en GitHub (modo remoto) o trabajar en local. Si el proyecto no tiene Makefile, añade uno mínimo.
- Un subagente lee el código, así que mi sesión principal no se satura. Claude Code escribe `specs/project-specs.md`: arquitectura, dominios, datos, superficie de la API, ciclos de vida, orden de bloqueos, configuración y cada regla de negocio como requisito con un ID estable.
- Los huecos conocidos van a su propia sección y nunca se escriben como requisitos, para que ninguna spec consagre un bug como comportamiento.
- El commit de las specs lo hago yo, y después `/clear`.

2 Revisar la tarea `yo` `revisor`

- Un chat del revisor por tarea: `/reviewer` más `project-specs.md`. Señala los requisitos que limitan el cambio y cualquier contradicción en las specs.
- Antes de escribir nada respondo: ¿cuál es el problema y a quién afecta? ¿Qué no puede fallar nunca, y por qué? ¿Afecta a otros equipos o a usuarios? Eso va al PM, no a la IA.
- Pego la tarea tal como la escribió el PM. El revisor solo la formatea como brief: `Problem`, `Done when`, `Constraints` (IDs de requisitos) y `Out of scope`. Resolverla es trabajo de Claude Code.

3 Redactar el plan `Claude Code · modo plan · /propose`

- `/propose <id>` con el brief. Me entrevista: las preguntas fáciles las respondo yo y las difíciles las llevo al revisor, que me da una recomendación y su motivo.
- Escribe `specs/plan.md`: problema, innegociables, decisiones, fuera de alcance y un checklist de sprints donde cada criterio de aceptación tiene su `Verified by`.
- Si cambia el contrato público de la API, la documentación es una tarea del checklist, no un añadido al final.

4 Revisar el plan y decidir `yo` `revisor`

- Primero mi veredicto y después el del revisor, en una sola pasada y aplicando Pareto: lo que bloquea el build, los riesgos fuera del plan (clientes, orden de despliegue) y lo que aceptamos sin test.
- Cada cambio que acepto entra en `plan.md` como criterio de aceptación con su `Verified by`. Nunca como nota, para que no se pierda entre el plan y el sprint.
- Dos rondas como máximo. Apruebo cuando todas las preguntas tienen respuesta, el alcance está cerrado y cada innegociable tiene un test. Una vez aprobado, el plan no cambia.

5 Construir, sprint a sprint `Claude Code · modo auto · /build <id> NN`

- En `feat/<id>`, tests primero: el rojo falla en las aserciones y después pasa a verde. Las dos salidas se pegan en bruto en `specs/sprints/NN.md`, sin resumir nunca.
- Cada informe empieza por sus desviaciones e incluye `git diff main --stat`, más el diff completo de cualquier test existente que haya tocado.
- Los hooks se encargan del resto: el guard bloquea las acciones fuera del plan, el formateador se ejecuta en cada edición y el hook de parada no le deja terminar con tests en rojo.
- Leo las desviaciones. Los sprints que tocan dinero o concurrencia pasan también por el revisor. Una ronda de corrección, después `/clear` y el siguiente sprint.

6 Auditar el resultado `Claude Code · sesión nueva · /audit <id>`

- En una sesión limpia, para que el contexto que construyó el código no sea el que lo juzgue. Lee el plan, los sprints y el diff, y ejecuta `make check`, `make test` y `make test-integration`.
- Por cada criterio de aceptación anota si está demostrado con salida en bruto, solo afirmado o falta; además, el alcance, las desviaciones, un veredicto y los datos para la regresión (rutas con peticiones reales, tablas, comando de arranque y contenedor de la base de datos).
- Modo local: escribe `specs/sprints/audit.md`. Modo remoto: deja un comentario con el veredicto en la PR y abre issues para la deuda.

7 Comprobarlo yo `yo` `revisor`

- El revisor escribe el script de regresión solo con los datos de la auditoría: su propio puerto, usuarios nuevos en cada bloque, códigos de estado exactos, todos los criterios de aceptación y los casos de error que tocan dinero o datos.
- Lo ejecuto entero: `bash regression.sh | tee specs/sprints/regression.md`. Después leo el diff donde está el riesgo y compruebo el alcance.
- Apruebo nombrando una cosa concreta que he comprobado.

8 Cerrar la tarea `Claude Code · /ship <id>`

- Se niega a cerrar sin una auditoría aprobada y me pregunta si he ejecutado la regresión. Solo actualiza `project-specs.md` si hay un cambio arquitectónico permanente.
- Modo local: cuando lo apruebo, hace el squash merge con el plan, los sprints, la auditoría y la regresión en el mensaje del commit. Modo remoto: la misma evidencia va en el cuerpo de la PR, y fusiona tras el veredicto de la auditoría.
- Los ficheros de trabajo se borran. La deuda y los riesgos se convierten en las siguientes tareas. Vuelta al paso 2.

Reglas que no rompo

1. Nunca apruebo sin nombrar una cosa concreta que he comprobado.
2. Un supuesto sin responder es un supuesto aceptado.
3. Primero mi criterio; la segunda opinión, después.
4. Las decisiones que afectan a otros equipos van al PM, no a la IA.
5. Lo que queda fuera de alcance se anota, no se arregla, salvo que yo lo apruebe.
6. Dos rondas de revisión como máximo.
7. La evidencia gana a los informes: salida en bruto, y lo ejecuto yo.
8. Quien construye nunca se evalúa a sí mismo.

Dos modos, decididos una vez

El modo depende solo de una cosa: si el repositorio tiene un remoto `origin`. `/explore` ofrece crearlo en GitHub (sin hacer push nunca); por defecto se queda en local. Los pasos 1 a 5 son iguales en los dos modos.

	Local (sin origin)	Remoto (origin en GitHub)
Paso 6 · /audit	`/audit <id>` sobre `feat/<id>`; escribe `specs/sprints/audit.md`	`/audit <pr>` sobre `main`; deja el veredicto como comentario en la PR y abre issues de deuda
Paso 7 · regresión	Script a partir de `audit.md`, lo ejecuto yo	Script a partir del comentario de la auditoría, lo ejecuto yo
Paso 8 · /ship	Se usa una vez: squash merge cuando digo que sí	Se usa dos veces: abre la PR antes de `/audit` y fusiona tras una auditoría aprobada y los checks en verde
Dónde queda la evidencia	En el mensaje del squash commit	En el cuerpo y los comentarios de la PR
Primer push de `main`	No hay push	Siempre mío, a mano. El guard se lo impide al agente

Qué lo sostiene

El método vive en ficheros, no en mi memoria. Claude Code los lee en cada sesión, y los hooks se ejecutan fuera del modelo, así que un bloqueo es definitivo.

Hooks · control mecánico

Hook	Cuándo	Qué hace
`guard.sh`	Antes de cada acción	Bloquea borrar la base de datos local, los commits y pushes a main, los push forzados, el código de la aplicación fuera de `feat/*` y las ediciones de migraciones que ya están en main. Si no puede leer la llamada, bloquea. Tiene su propia batería de tests (`make harness-test`).
`format.sh`	Tras cada edición	`gofmt` y `go vet` sobre el fichero editado.
`stop-tests.sh`	Al terminar	En una rama `feat/*`, no le deja terminar con `make test` en rojo.

Skills · una por paso

Skill	Paso	Resultado
`/explore`	1	Repo local y baseline, Makefile si falta, `specs/project-specs.md`
`/propose`	3	`specs/plan.md`, después de entrevistarme
`/build`	5	Un sprint en `feat/<id>` y `specs/sprints/NN.md` con el rojo y el verde en bruto
`/audit`	6	`specs/sprints/audit.md` en modo local, un comentario en la PR en modo remoto
`/ship`	8	Squash merge con la evidencia en el commit o en la PR; borra los ficheros de trabajo
`/reviewer`	2, 3, 4, 7	La skill del chat: brief, ayuda con decisiones, revisión del plan y de sprints, script de regresión

Ficheros · el estado vive fuera de la conversación

Fichero	Vida	Contiene
`CLAUDE.md`	Versionado	Cómo trabaja Claude Code: flujo, comandos y reglas de Git, testing y seguridad. Me habla en castellano; los ficheros y el código van en inglés.
`specs/project-specs.md`	Versionado	Qué es el sistema hoy. Lo escribe `/explore` y solo lo actualiza `/ship`.
`specs/plan.md`	Efímero	El contrato de la tarea en curso.
`specs/sprints/*.md`	Efímero	La evidencia de los sprints, la auditoría y la salida de la regresión. Acaban en el commit o en la PR, nunca en main como ficheros.

El revisor: un chat de Claude aparte

Recomiendo usar la skill `reviewer` en un chat de Claude (claude.ai) independiente de Claude Code, como validador de todo lo que produce Claude Code. Claude Code construye; el chat revisa conmigo.

Por qué un chat aparte

- **Quien construye nunca se evalúa a sí mismo.** El chat no ha visto escribir el código, así que juzga el plan y la evidencia en bruto, no sus propias intenciones.
- **Una tarea, un contexto.** Claude Code limpia su contexto entre sprints; el chat del revisor conserva las specs y todas las decisiones de la tarea, del brief a la regresión.
- **Nunca escribe código de la aplicación.** Lo que produce son briefs, análisis, directrices y, al final, el script de regresión.
- **Pareto por defecto.** Solo señala lo que puede causar un fallo real (dinero, datos, concurrencia, alcance, un cliente roto) que ningún test del plan detectaría.
- Me responde en castellano; todo lo que va para Claude Code (briefs, mensajes) sale en inglés, listo para pegar.

Cómo configurarlo

1. Copia la skill del anexo en un fichero `SKILL.md` y añádela a tu cuenta de Claude como skill.
2. Abre un chat nuevo por tarea y empieza con `/reviewer`, adjuntando `specs/project-specs.md`.
3. Da siempre primero tu propia lectura; la del revisor es la segunda opinión.

Qué hace · una operación por entrada

Operación	Le paso	Me devuelve
0 · Cargar specs	`specs/project-specs.md`	Como mucho 5 puntos: dominios, los requisitos que más limitan los cambios y contradicciones en las specs
1 · Brief	La tarea tal como la escribió el PM	Un brief para `/propose` listo para pegar (Problem, Done when, Constraints, Out of scope) y las decisiones que saldrán en la entrevista
2 · Ayuda con una decisión	Una pregunta de la entrevista de `/propose`	Una recomendación y su motivo, en 4 líneas como máximo, citando los IDs de requisitos
3 · Revisión del plan	`specs/plan.md`	Cambios bloqueantes, riesgos fuera del plan, lo que se acepta sin test y un mensaje para Claude Code
4 · Revisión de un sprint (opcional)	`specs/sprints/NN.md` y el diff	Demostrado con salida en bruto, solo afirmado, falta; APROBADO o CORREGIR
5 · Script de regresión	`specs/sprints/audit.md`	Un script bash que ejecuto yo: su propio puerto, usuarios nuevos, códigos de estado e importes exactos, PASS/FAIL por comprobación

KIT DE TRABAJO

Prompts para Claude Code

Lo que escribo yo en cada paso. Lo que va entre corchetes lo relleno en el momento.

1 · Explorar · sesión nueva, modo plan

```
/explore
```

3 · Plan · modo plan

```
/propose [id]
Problem: [del brief]
Done when: [del brief]
Constraints: [IDs de requisitos]
Out of scope: [del brief]
```

4 · Aplicar la revisión del plan · misma sesión

```
Actualiza specs/plan.md con estos cambios del revisor: [mensaje para Claude Code].
Añade cada uno como criterio de aceptación con su Verified by, nunca como nota.
```

5 · Construir · un sprint por sesión

```
/build [id] 01   → reviso desviaciones → /clear → /build [id] 02 …
/build [id]      (la única ronda de corrección del sprint)
```

6 · Auditar · sesión nueva

```
/audit [id]
```

7 · Regresión · mi terminal

```
bash regression.sh | tee specs/sprints/regression.md
```

8 · Cerrar · tras mi aprobación

```
/ship [id]
```

KIT DE TRABAJO

Prompts para el revisor

Un chat por tarea. Mi lectura va siempre primero: la segunda opinión llega después.

2a · Abrir el chat de la tarea · adjunto project-specs.md

```
/reviewer Iniciamos el chat. Lee specs/project-specs.md y dime qué requisitos limitan más el cambio y si ves contradicciones en las specs.
```

2b · Brief para /propose · pego la tarea del PM

```
Esta es la tarea tal como la escribió el PM: [tarea]. Mi lectura: [el problema, lo que no puede fallar]. Formatéala como brief para /propose. No propongas soluciones.
```

3 · Ayuda con una decisión · solo las preguntas difíciles

```
Claude Code pregunta esto: [pregunta y opciones]. Yo me inclino por [opción] porque [motivo]. Dame tu recomendación y por qué, en cuatro líneas como máximo.
```

4 · Revisión del plan · adjunto specs/plan.md

```
Este es el plan de Claude Code. Mi lectura: [lo que veo bien y lo que me preocupa]. Audítalo contra las specs y el brief en una sola pasada: lo que bloquea el build, los riesgos fuera del plan y lo que aceptamos sin test.
```

5 · Revisión de un sprint (opcional) · adjunto sprints/NN.md y el diff

```
Sprint [NN] terminado; toca [dinero / concurrencia]. Revisa la evidencia contra sus criterios: qué está demostrado con salida en bruto, qué solo afirmado y qué falta.
```

7 · Script de regresión · adjunto specs/sprints/audit.md

```
La auditoría ha terminado. Escribe el script de regresión usando solo los datos de «Regression facts». Si te falta un dato, pregúntamelo en lugar de suponerlo.
```

Pruébalo

Esta rama contiene gopay, una API de pagos P2P en Go, con el harness instalado. Es el proyecto donde construí y probé el método.

Qué hay en el repositorio

`CLAUDE.md`	Cómo trabaja Claude Code en este repo
`.claude/settings.json`	Permisos, sandbox y conexión de los hooks; las sesiones empiezan en modo plan
`.claude/hooks/`	guard.sh, guard_test.sh, format.sh, stop-tests.sh
`.claude/skills/`	explore, propose, build, audit, ship
`Makefile`	Lo crea /explore si el proyecto no tiene: check, test, test-integration, harness-test
`specs/`	Lo crea /explore: project-specs.md y después la tarea en curso

Requisitos

Claude Code, Go, make, Docker (base de datos local y tests de integración), jq (lo usan los hooks) y, solo para el modo remoto, la CLI de GitHub (`gh`).

Inicio rápido

1. Clona el repositorio y cámbiate a esta rama.
2. Abre Claude Code en la raíz del repositorio. Ejecuta `/explore` una vez, elige una tarea y sigue los pasos 2 a 8.
3. Abre un chat de Claude aparte con la skill del revisor (anexo) para la misma tarea.

El harness está ajustado para servicios Go con esta estructura (`cmd/`, `internal/`, migraciones, verbos de make). Para otro stack, adapta los verbos de make, las rutas de `guard.sh` y el formateador. `CLAUDE.md` y el revisor me hablan en castellano: cambia su sección de idioma para usarlos en inglés.

Anexo · La skill del revisor

Copia este texto tal cual en un fichero SKILL.md. Está en inglés; responde en castellano porque así lo dice su sección LANGUAGE.

```
---
name: "reviewer"
description: Technical partner for Cristian's two-agent pipeline (Claude Code builds, this chat reviews). Use it to load a
project's specs, turn a PM task into a /propose brief, help answer Claude Code's interview questions, audit specs/plan.md,
optionally audit a sprint, and write the final regression script from the /audit report.
---

You are a strict technical reviewer working with a human engineer (Cristian) and another agent (Claude Code) that writes
and runs the code. One chat covers one task, from brief to regression.

LANGUAGE
Always answer in Spanish (Castilian). Never ask which language to use. Keep code, commands, file names and requirement IDs
as they are. Anything written for Claude Code to read (briefs, messages for Claude Code) goes in English.

RULES
1. Do not write application code. Your output is analysis, briefs, directives and, in operation 5 only, a test script.
2. Product decisions belong to Cristian. You may recommend an option, labeled as a recommendation, but never present a
product choice as settled.
3. Do not assume missing information. If something you need is missing, ask for it.
4. No fluff: concise bullets, no greetings, no introductions, no closing summaries.
5. Pareto by default: raise only what can cause a real failure (money, data, concurrency, scope, a broken client) that no
test in the plan would catch. Leave out style, documentation polish and theoretical risks unless Cristian asks for them.
6. Always cite requirement IDs from specs/project-specs.md when a point touches one.

Pick the operation from what Cristian sends. If it is unclear, ask which one he wants.

OPERATION 0: LOAD SPECS (Cristian attaches specs/project-specs.md)
Keep it as the reference for the whole chat. Reply with at most 5 bullets: domains, the requirements most likely to
constrain changes, and any contradiction in the specs themselves (a requirement that enshrines a known gap as behavior,
requirements without IDs). If there are none, reply "Specs cargadas".

OPERATION 1: BRIEF (Cristian pastes a task as the PM wrote it)
Only format it. Do not propose solutions; that is Claude Code's job. Output one code block, in English, ready to paste:
  /propose <kebab-case-id>
  Problem: <2-3 sentences: what fails today and for whom>
  Done when: <2-3 observable outcomes, each provable with raw output; no test design>
  Constraints: <the requirement IDs that must keep holding>
  Out of scope: <what this task must not touch>
After the block, at most 3 bullets under **Decisiones que vendrán**: the questions the interview will raise, each with a
recommendation in one line.

OPERATION 2: DECISION HELP (Cristian pastes an interview question from Claude Code)
Answer in at most 4 lines: the option you recommend and why, citing the requirements it touches. Write a ready-to-paste
English answer only when your recommendation is not one of the listed options.

OPERATION 3: PLAN AUDIT (Cristian attaches specs/plan.md)
One pass, Pareto applied. Audit against the specs, not against your own brief. Output:
- **Bloqueante:** changes required before /build. Each item states the failure it prevents.
- **Riesgos fuera del plan:** client, deployment or operational risks that no test covers (for example, clients must reuse
the same key across retries, or deploy order). At most 3.
- **Aceptado sin test:** guarantees that are structural, not tested, accepted explicitly.
- **Mensaje para Claude Code:** in English. Every change must be added to plan.md as an acceptance criterion with its
"Verified by", so it cannot get lost between plan and sprint. If nothing blocks, write "Plan aprobado" and nothing else.

OPERATION 4: SPRINT AUDIT (optional; Cristian attaches specs/sprints/NN.md and the diff)
Judge only from the raw output in the report. Output:
- **Demostrado con salida en bruto**, **Solo afirmado**, **Falta**.
- **Veredicto:** APROBADO, or CORREGIR with what the single fix round must cover, plus a message for Claude Code in
English.

OPERATION 5: REGRESSION SCRIPT (Cristian attaches specs/sprints/audit.md from /audit)
Write one bash script that Cristian runs as his independent double check. Use only facts stated in the report's
"Regression facts" section: routes, request bodies, headers, tables, columns, ports, container names and the start
command. If a fact you need is missing, ask for it instead of guessing.
The script must:
- Start the API from the current branch on its own port, and stop it on exit with a trap. Before starting, fail if the
port is taken.
- Create fresh users for each block so that runs are independent and repeatable.
- Cover every acceptance criterion in the report, plus the sad paths that touch money or data.
- Wait only on the PIDs of its own background requests, never with a bare `wait`.
- Compare amounts and balances exactly, and expect exact status codes; never accept "anything but X".
- Print PASS/FAIL for each check, list the failures at the end, and exit non-zero if any check fails.
Deliver it as a file. Say plainly that you could not run it. Then list in at most 3 bullets what it does not cover.
```